Midterm Exam

● Graded

Student

Neal Sharma

Total Points

95 / 100 pts

Question 1

(no title) **14** / 16 pts

1.1 **(no title)** **2** / 2 pts

✔ **– 0 pts** Correct answer I.

1.2 **(no title)** **0** / 2 pts

✔ **– 2 pts** The address space relates to the organization of memory, it is not related to scheduling. For multitasking, we need a scheduler that has preemption (answer D).

1.3 **(no title)** **2** / 2 pts

✔ **– 0 pts** Correct answer is J.

1.4 **(no title)** **2** / 2 pts

✔ **– 0 pts** Best answer is L.

1.5 **(no title)** **2** / 2 pts

✔ **– 0 pts** B is the correct answer.

1.6 **(no title)** **2** / 2 pts

✔ **– 0 pts** E is the correct answer.

1.7 **(no title)** **2** / 2 pts

✔ **– 0 pts** Heap is the correct answer.

1.8 **(no title)** **2** / 2 pts

✔ **– 0 pts** Correct

Question 2

(no title) **30** / 30 pts

2.1 **(no title)** **2** / 2 pts

✔ **– 0 pts** False is the correct answer.

2.2 **(no title)** **2** / 2 pts

✔ **– 0 pts** True is correct.

2.3 **(no title)** **2** / 2 pts

✔ **– 0 pts** True is the correct answer.

2.4 **(no title)** **2** / 2 pts

✔ **– 0 pts** False is the correct answer.

2.5 **(no title)** **2** / 2 pts

✔ **– 0 pts** True is the correct answer.

2.6 **(no title)** **2** / 2 pts

✔ **– 0 pts** False is the correct answer.

2.7 **(no title)** **2** / 2 pts

✔ **– 0 pts** False is the correct answer.

2.8 **(no title)** **2** / 2 pts

✔ **– 0 pts** Correct

2.9 **(no title)** **2** / 2 pts

✔ **– 0 pts** True is the correct answer.

2.10 **(no title)** **2** / 2 pts

✔ **– 0 pts** True is the correct answer.

2.11 **(no title)** **2** / 2 pts

✔ **– 0 pts** False is the correct answer.

2.12 **(no title)** **2** / 2 pts

✔ **– 0 pts** False is the correct answer.

2.13 **(no title)** **2** / 2 pts

✔ **– 0 pts** True is the correct answer.

2.14 **(no title)** **2** / 2 pts

✔ **– 0 pts** True is the correct answer.

2.15 **(no title)** **2** / 2 pts

✔ **– 0 pts** True is the correct answer.

Question 3

(no title) **14** / 16 pts

3.1 **(no title)** **3** / 4 pts

✔ **– 1 pt** Not dividing by 3 to find average.

3.2 **(no title)** **3** / 4 pts

✔ **– 1 pt** Not dividing by 3 to get the average.

3.3 **(no title)** **4** / 4 pts

✔ **– 0 pts** Correct

3.4 **(no title)** **4** / 4 pts

✔ **– 0 pts** Correct

Question 4

(no title) **12** / 12 pts

✔ **– 0 pts** Number of page faults is 6 and work is shown.

Question 5

(no title) **12** / 12 pts

✔ **– 0 pts** 2268 is correct final answer
or formula is correct calculation
Example: 2*1024+(1244%1024)

Question 6

(no title) **13** / 14 pts

6.1 (no title) **5** / 5 pts

✔ **– 0 pts** Expect to see something like the following. Grader expects to see at least two of the bolded terms. But solutions that have significant wrong or unnecessary writing (aka "fishing for an answer" by copying junk from a cheat sheet and not directly answering the question) will lost some points.

sys_boost() can use **privileged instructions** and access **kernel** memory because it runs in **kernel mode**. Specifically, sys_boost() will need to change the priority in the **proc** table, boost(pid) would not be able to do this because it runs in **user mode** and does not have access to the table in **kernel memory**.

6.2 (no title) **5** / 5 pts

✔ **– 0 pts** Explanation is clear and complete and includes at least 3 for the following key terms:
system call
privileged instruction or **ecall**
trap or **interrupt**
interrupt handler
return from interrupt

6.3 (no title) **3** / 4 pts

✔ **– 1 pt** Says that the parameter is stored by boost(pid) in a register and read by sys_boost() from the register. This answer is confusing the system call id (which is passed by register) with function arguments (which are passed on the call stack).

This answer shows understanding, but is not fully correct or complete. We want to see mention of the **trapframe** or **call stack** or **stack** (all acceptable) for full credit. sys_boost() cannot take parameters from boost(pid) because it is being called from a **trap/interrupt handler**, not a **direct function call** from boost(pid).

COM S 352: Introduction to Operating Systems
Midterm Exam
Fall 2026

Cover Sheet

Student Name: Neal Sharma

Format:

- Time: 75 minutes
- Points: 100
- Question Types: matching, true/false and short answer

Instructions:

- You may use 1 (one) letter-sized sheet of paper (front and back), that you have prepared yourself with notes before the exam, as a "cheat sheet" during the exam.
- You may not consult classmates, electronic devices or resources other than the cheat sheet during the exam.
- Questions of clarification should be asked directly to an instructor or TA.

Question	Points
1	/16
2	/30
3	/16
4	/12
5	/12
6	/14
Total	/100

1. (16 pts max, 2 pts each) For each description on the left, select *the best* matching term on the right, each term is used only once but some will not be used.

I fork — the system call that creates a new process
C Address Space — required to implement multitasking
J stride — a scheduling algorithm that provides fairness
L MMU — responsible for translating virtual to physical address
B Parent Process — identified by fork() returning greater than zero
E Page Out — moving a page from a frame to a block
F heap — one of the common segments in a process
K Thrashing — caused by too much multitasking

A. page table
B. parent process
C. address space
D. preemption
E. page out
F. heap
G. zombie
H. privileged instruction
I. fork
J. stride
K. thrashing
L. MMU
M. segmentation

2. (30 pts max, 2 pts each) Which of the following statements are true? Write T or F for true or false.

F System calls are used in place of regular function calls because they provide faster access to the kernel.

T OPT is a replacement policy that requires knowing future memory accesses.

T In MLFQ, a job that uses its entire time slice moves down a queue.

F When using a RR (round robin) scheduling policy, it is possible for a process to prevent other processes from running by staying on the CPU for as long as it wants.

T A system that is thrashing is experiencing a high level of page outs and page ins.

F A benefit of larger page size is a reduction in internal fragmentation.

F The dirty bit in a page table entry is set whenever the process attempts to access a page it doesn't have permission to access.

F The CFS scheduler picks the process with the smallest real runtime to run next.

T The page table needs to be updated when handling a page fault.

T Swap and TLB can use the same replacement policies.

F When a user program executes a system call, it switches from kernel mode to user mode.

F Changing a process from the blocked state to the ready state is an example of a context switch.

F Loops are a source of high temporal locality in programs.

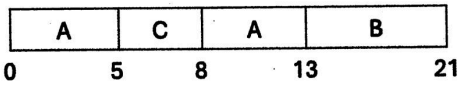
T When a process is in the ready state it is waiting for CPU time and not for any I/O operation to complete.

T The clock algorithm replacement policy takes advantage of a feature found on some computer hardware.

3. (16 pts) Consider the following set of jobs, with arrival times and the length of CPU bursts (job runtimes) given in milliseconds.

	Arrival Time	CPU Burst
A	0	10
B	4	8
C	5	3

Suppose a scheduler produces the schedule shown in the Gantt chart below.



a) The average turnaround time is (you must show work, can leave answer as fraction):

Turnaround : Finish - Arrival

$\Rightarrow \left((13-0)+(21-4)+(8-5)\right)/4 \Rightarrow \frac{13+17+3}{4} = \frac{33}{4}$

b) The average response time is (you must show work, can leave answer as fraction):

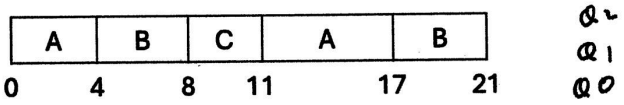
Response : first run - arrival

$\Rightarrow (0-0)+(13-4)+(5-5))/4 \Rightarrow \frac{9}{4}$

c) The given schedule was most likely produced by which of the following scheduling policies: ~~FIFO~~, ~~SJF~~, STCF, ~~RR~~ (choose **one and only one** of the given options).

STCF

d) Now suppose a Multi-Level Feedback Queue (MLFQ) produces this alternative schedule from the original set of jobs:



Assume the MLFQ has 3 levels labeled Q2 (highest priority with time quantum of 4), Q1 (middle priority with time quantum of 8), and Q0 (lowest priority with time quantum of 16).

The process for job A ends in queue Q 1

The process for job B ends in queue Q 1

The process for job C ends in queue Q 2

4. (12 pts) Given the reference string of page accesses: 9 8 1 2 9 1 8 9 and a system with 3 page frames, how many page faults result when using the LRU (least recently used) replacement policy? Show your work. LRU 3 frames

The number of page faults is 6 (show work below)

	9	8	1	2	9	1	8	9
Evict				9	8		2	
Least recently used			9	8	1	2	9	1
		9	8	1	2	9	1	8
most recently used	9	8	1	2	9	1	8	9
	M	M	M	M	M	H	M	H
	1	2	3	4	5		6	

5. (12 pts) Consider the page table below. Assume a page size of 1KB (1,024 bytes).

Page Table

VPN	PFN	Valid
0	4	1
1	2	1
2	1	1
3	0	0
4	0	0
5	8	1
6	12	1

Reference table of conversions:

1KB	1,024 bytes
2KB	2,048 bytes
3KB	3,072 bytes
4KB	4,096 bytes
5KB	5,120 bytes
6KB	6,144 bytes
7KB	7,168 bytes
8KB	8,192 bytes
9KB	9,216 bytes
10KB	10,240 bytes

1244
-1024
220

What is the physical address in memory that will be read when a program reads a byte from virtual address 1,244?

Offset = 1244 - 1024 ⇒ 220 ; 8 1KB page size,

∴ It will lee in VPN 1 = PFN 2

Base address of PFN 2 ⇒ 2 × 1024 ⇒ 2048

Now add offset ⇒ 2048 + 220

Physical address ⇒ 2268

6. (14 pts) Suppose the xv6 scheduler has been modified to be a priority-based scheduler and now we want to add a new system call: `boost`, which allows a user to give a chosen process the highest priority level. First, we create a function on the user-side: `boost(pid)`, where `pid` is the process id of the process set to the highest priority, and add it to `user/user.h`. In the kernel-side code we create the function `sys_boost()` and add it to `kernel/sysproc.c`. Answer the following questions.

a) What can `sys_boost()` do that `boost(pid)` cannot do? Specifically, why is it not possible to put all the code for boosting a process priority level into `boost(pid)`?

sys_boost() is kernal level while boost(Pid) is user level. It can perform elevated (unrestricted) operations. So here boost(Pid) causes a trap to switch to the kernel level sys_boost() which elevates the priority of the chosen process. Boost(Pid) can't access processes / memory outside its address space of the program.

b) When the user calls `boost(pid)` how does `sys_boost()` get called? Give a detailed description of the events that need to occur. ∴ Need sys_boost()

boost(Pid) performs a system call initiated by causing a trap which hands control to sys_boost() which is kernel level [control transferred to OS]. The trap causes it to jump to the OS trap handler. CPU context is saved in a stored procedure

c) `boost(pid)` takes one parameter, however, `sys_boost()` takes none. How does `sys_boost()` know what `pid` to use? Why are these not parameters of `sys_boost()`?

This is because →
sys_boost is a kernel side code and has access to the program's registers, stack, heap. It can access the memory of the calling process, via the trap handler and the sys_boost() code.